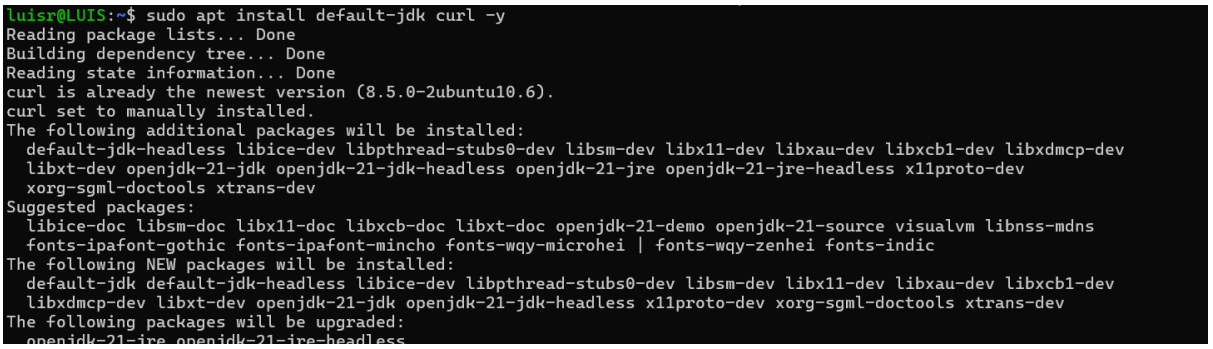
CO1 FA3 Spark

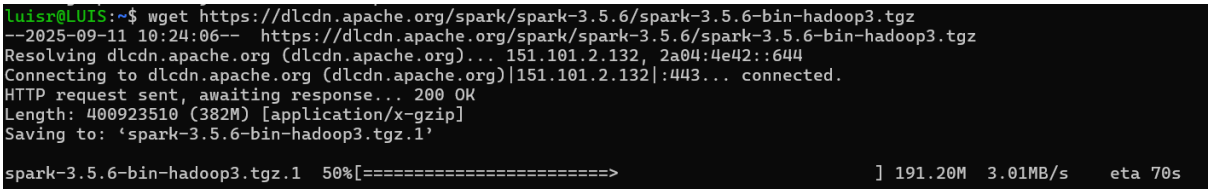
Members: Dela Cruz, Cloie Sofia & Respecio, Luis Arnold N.

Install Java and Utilities



Installing Apache Spark requires Java to execute its runtime processes. Along with Java, we also install `curl` so we can download things from the internet via command-line. `curl` is a utility that transfers data from or to a server using many protocols (like HTTP, HTTPS, FTP). It is widely used because it's simple and already available on many Linux systems, and works well for scripts and automation. (GeeksforGeeks, 2025)

Download and Extract Spark



To install Apache Spark, we need to download the packages from the official Apache servers. We will use the `wget` utility, a command to fetch the file directly from the web into our current directory. (GeeksforGeeks, 2024) This command will download the Spark 2.5.6 version bundled with Hadoop 3.

Extract the Spark Archive

```
luisr@LUIS:~$ tar -xzvf spark-3.5.6-bin-hadoop3.tgz
spark-3.5.6-bin-hadoop3/
spark-3.5.6-bin-hadoop3/examples/
spark-3.5.6-bin-hadoop3/examples/jars/
spark-3.5.6-bin-hadoop3/examples/jars/spark-examples_2.12-3.5.6.jar
spark-3.5.6-bin-hadoop3/examples/jars/scopt_2.12-3.7.1.jar
spark-3.5.6-bin-hadoop3/examples/src/
spark-3.5.6-bin-hadoop3/examples/src/main/
spark-3.5.6-bin-hadoop3/examples/src/main/java/
spark-3.5.6-bin-hadoop3/examples/src/main/java/org/
spark-3.5.6-bin-hadoop3/examples/src/main/java/org/apache/
spark-3.5.6-bin-hadoop3/examples/src/main/java/org/apache/spark/
```

The file spark-3.5.6-bin-hadoop3.tgz will appear in your current directory which is a compressed archive which we will extract to make Spark files accessible. Since the Spark package is compressed, it needs to be extracted before we can use it.

The `tar` command is commonly used in Linux for handling such archives. (GeeksforGeeks, 2025) The above command will let us unpack the contents into a new folder called `spark-3.5.6-bin-hadoop3` which contains all the Spark files.

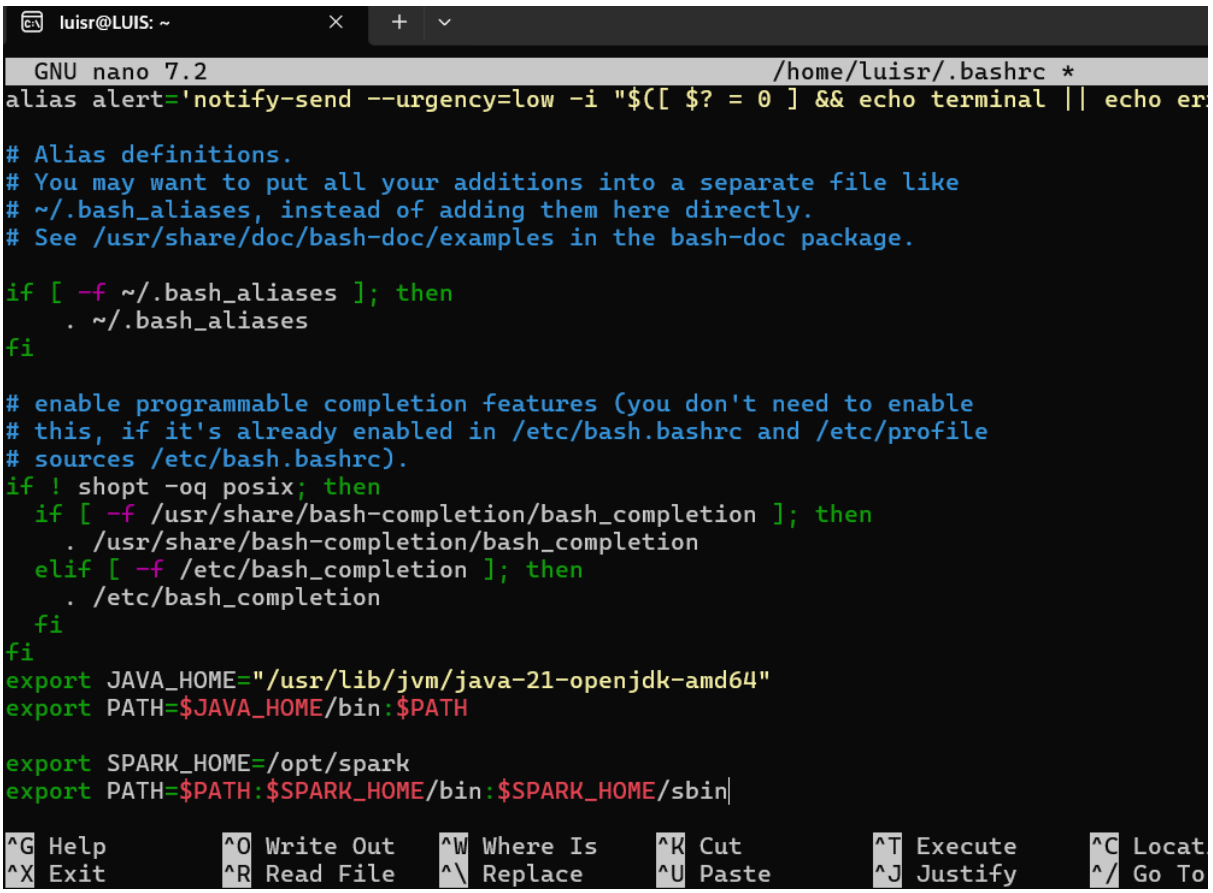
Move Spark to /opt/ Directory

```
luisr@LUIS:~$
luisr@LUIS:~$ sudo mv spark-3.5.6-bin-hadoop3 /opt/spark
luisr@LUIS:~$
```

For convenience and better organization, we move the extracted Spark folder to opt/spark.

Configure Environment Variables

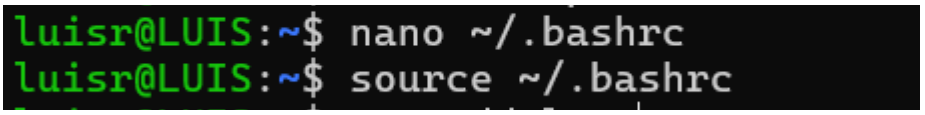
We now need to set environment variables so the system recognizes Spark commands globally. As said in the Linux documentation, the `.bashrc` file configures the shell environment and executes custom settings whenever a new session is started (Verma et al., 2025). We use the nano text editor to open it: `nano ~/.bashrc`



We now add Spark to the system's environment variables so that we can run Spark commands from any directory. Add the following at the bottom:

```
export SPARK_HOME=/opt/spark
```

```
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin
```



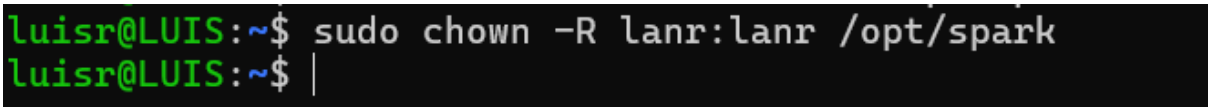
Then apply changes by `source ~/.bashrc`. The `source` command reloads the file, applying the changes without requiring restart. (GeeksforGeeks, 2025)

Create a Spark User

It is good practice not to run Spark as root. Instead, we create a dedicated user and give them ownership of Spark's files.



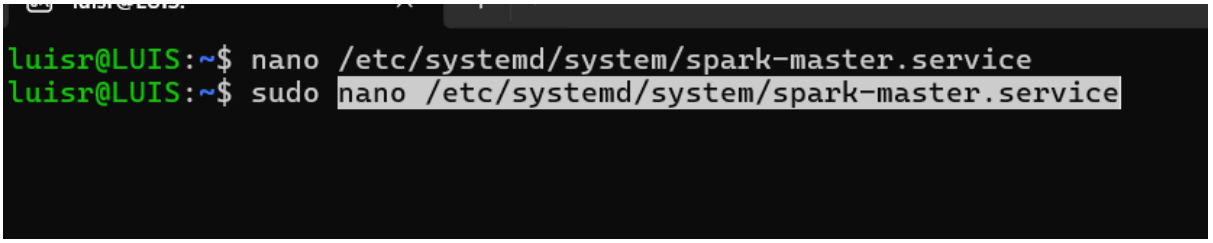
Here we create a new user as it is recommended to run Spark under a dedicated user. Replace lanr with your own username.



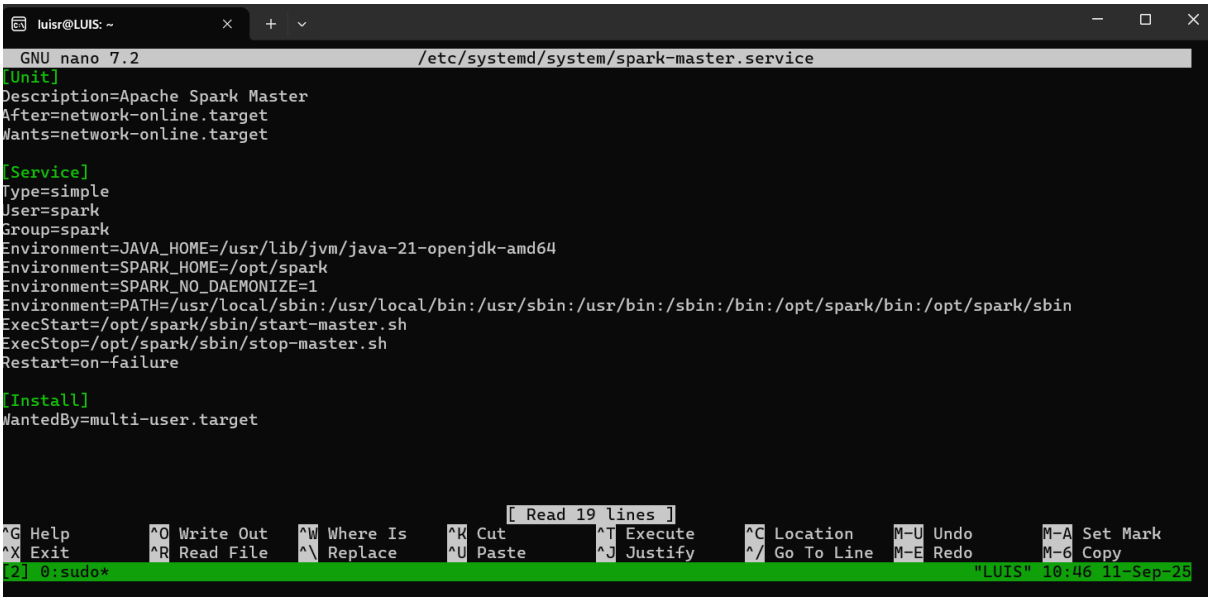
After creating a new user, we will give them ownership of the Spark directory.

Configure Spark Master as a System Service

In this step, we will set up the Spark Master to run continuously in the background as a managed service. This makes it easier to start, stop, and monitor Spark without manually launching it every time.



To start, we need to create a new service file.



Once the file is open, we add the configuration shown below:

[Unit]

Description=Apache Spark Master

After=network.target

[Service]

Type=forking

User=lanr ***//change this according to your set useradd.***

Group=lanr ***//change this according to your set useradd.***

ExecStart=/opt/spark/sbin/start-master.sh

ExecStop=/opt/spark/sbin/stop-master.sh

[Install]

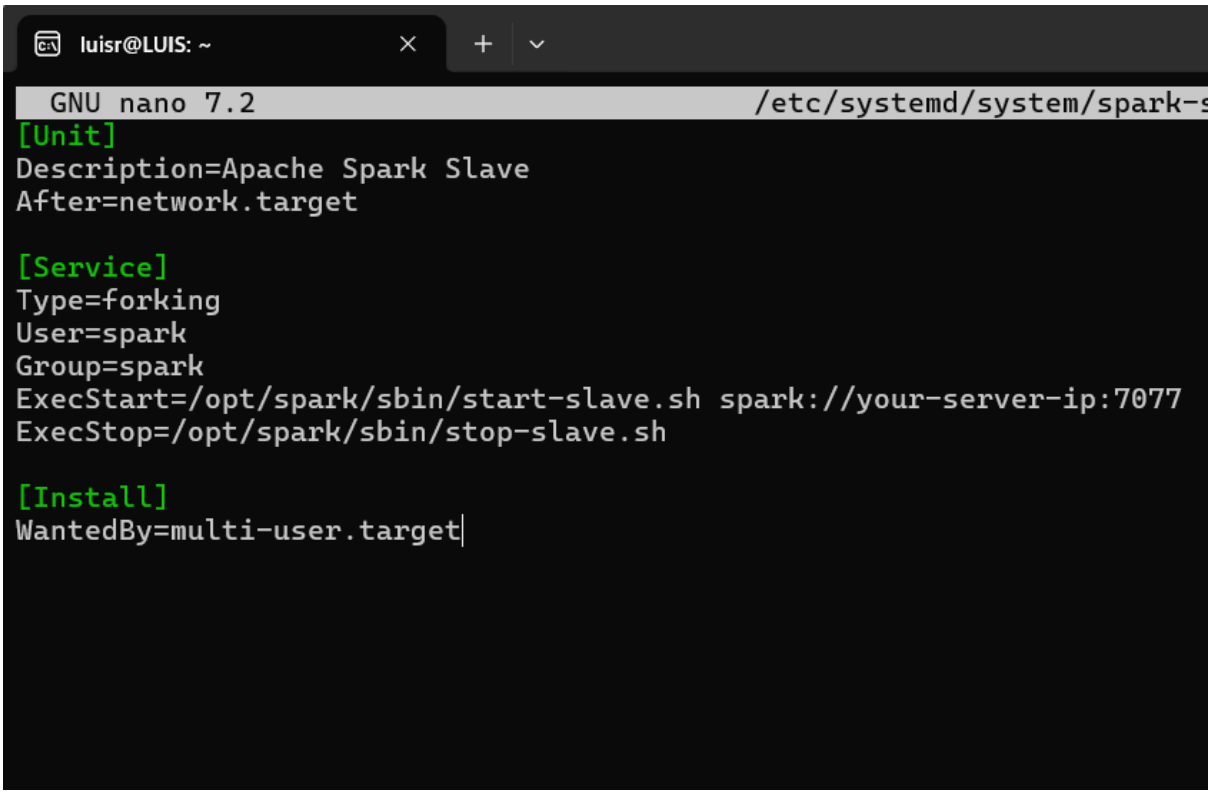
WantedBy=multi-user.target

Configure Spark Worker (Slave) as a System Service

In this step, we will set up the Spark Slave to run continuously in the background as a managed service. This makes it easier to start, stop, and monitor Spark without manually launching it every time.

```
luisr@LUIS:~$ sudo nano /etc/systemd/system/spark-slave.service
luisr@LUIS:~$
```

To start, we need to create a new service file.



Once the file is open, we add the configuration shown below:

[Unit]
Description=Apache Spark Worker
After=network.target

[Service]
Type=forking

User=lanr ***//change this according to your set useradd.***

Group=lanr ***//change this according to your set useradd.***

ExecStart=/opt/spark/sbin/start-worker.sh spark://<your-server-ip>:7077

ExecStop=/opt/spark/sbin/stop-worker.sh

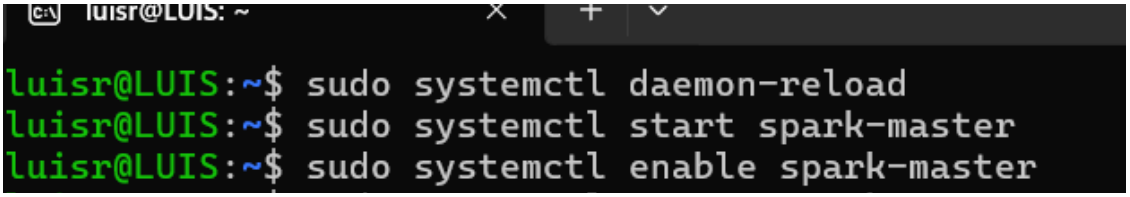
[Install]

WantedBy=multi-user.target

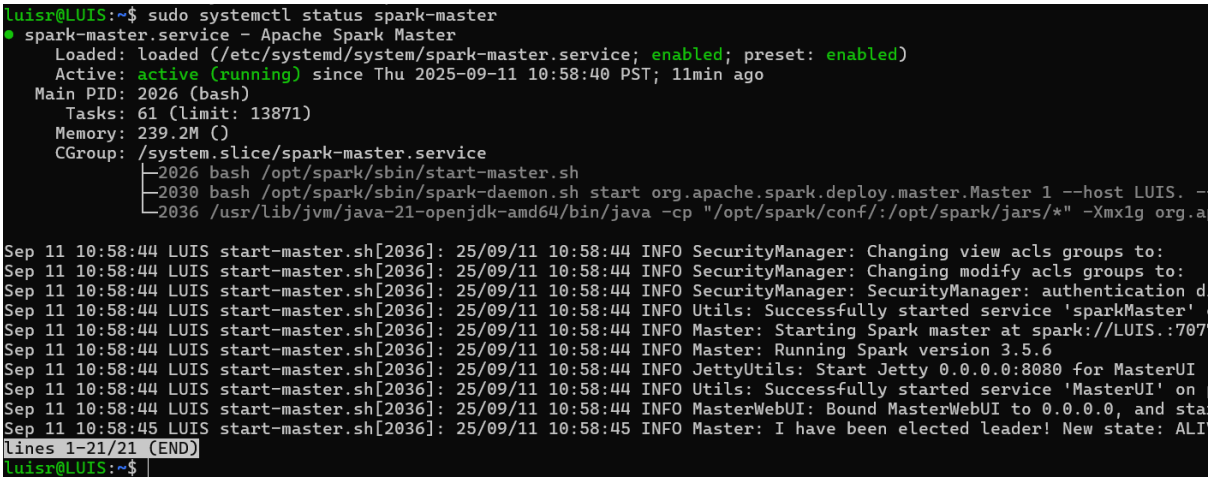
Enable and start Spark Master and Slave (Worker)

a. spark-master

Enable and start the spark-master service by running the following commands:



Check the status using `systemctl status spark-master.`



The status should show active (running) if successful.

b. spark-slave

Enable and start the spark-slave service by running the following commands:

```
luisr@LUIS:~$ sudo nano /etc/systemd/system/spark-worker.service
luisr@LUIS:~$ sudo systemctl daemon-reload
luisr@LUIS:~$ sudo systemctl enable spark-worker
luisr@LUIS:~$ sudo systemctl start spark-worker
```

// Some versions require spark-worker instead of spark-slave.

Check the status using `systemctl status spark-worker.`

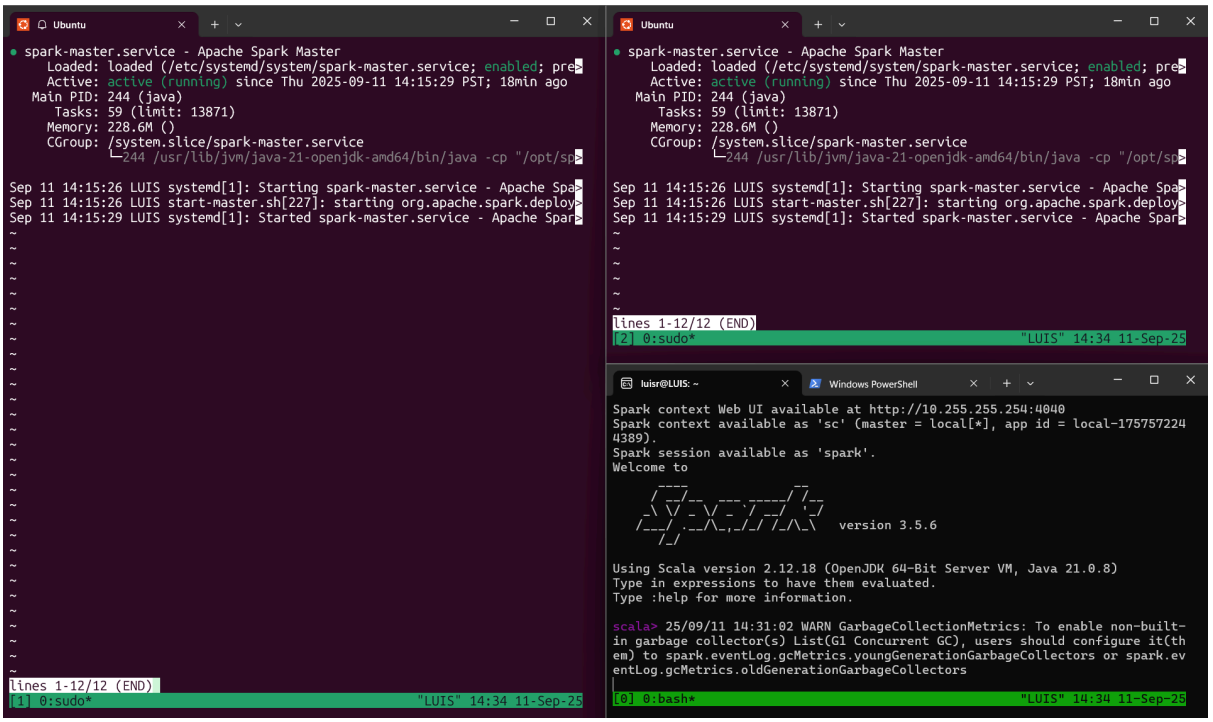
```
luisr@LUIS:~$ systemctl status spark-worker
● spark-worker.service - Apache Spark Slave
     Loaded: loaded (/etc/systemd/system/spark-worker.service; enabled; preset: enabled)
     Active: active (running) since Thu 2025-09-11 14:27:16 PST; 21s ago
    Process: 903 ExecStart=/opt/spark/sbin/start-worker.sh spark://your-server-ip:7077 (code=exited, status=0/SUCCESS)
   Main PID: 923 (java)
      Tasks: 65 (limit: 13871)
     Memory: 278.8M ()
     CGroup: /system.slice/spark-worker.service
             └─923 /usr/lib/jvm/java-21-openjdk-amd64/bin/java -cp "/opt/spark/conf/:/opt/spark/jars/*" -Xmx1g org.apac>

Sep 11 14:27:13 LUIS systemd[1]: Starting spark-worker.service - Apache Spark Slave...
Sep 11 14:27:13 LUIS start-worker.sh[907]: starting org.apache.spark.deploy.worker.Worker, logging to /opt/spark/logs/s>
Sep 11 14:27:16 LUIS systemd[1]: Started spark-worker.service - Apache Spark Slave.
lines 1-13/13 (END)
[0] 0:systemctl*                                                                    "LUIS" 14:27 11-Sep-25
```

Run Spark Shell

To confirm the installation, we open Spark in interactive mode:

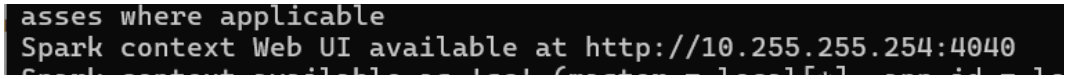
`spark-shell`



If successful, Spark Shell will start in the terminal, showing (scala>) prompts.

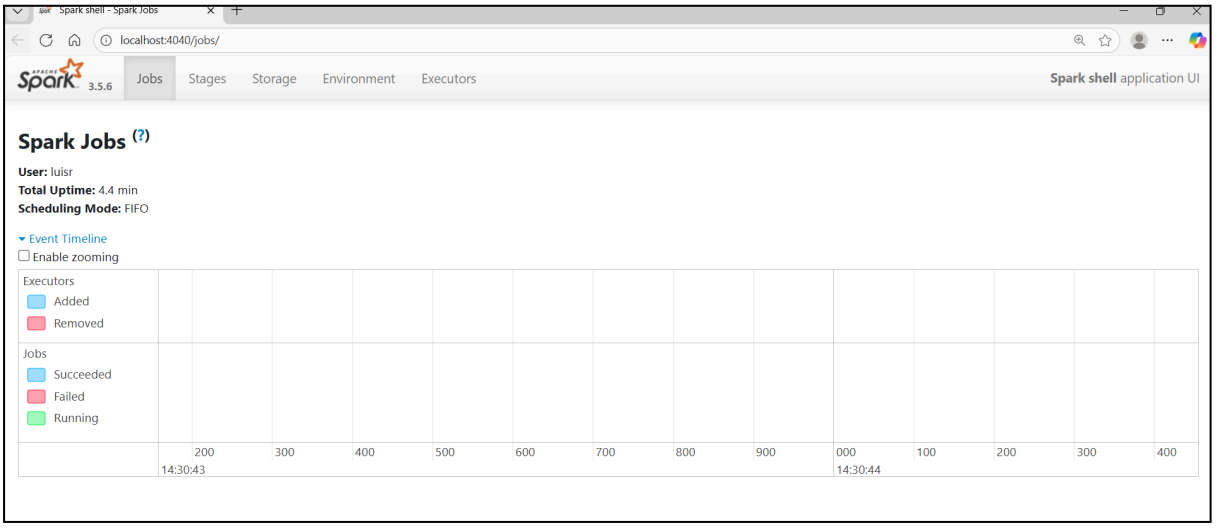
Explore the Spark Shell User Interface

When Spark Shell is running, you can CTRL + Left Click on the `http://localhost:4040`. This web UI helps users keep track of what Spark is doing in real time. The three most important sections are the **Jobs**, **Environment**, and **Executors** tabs.



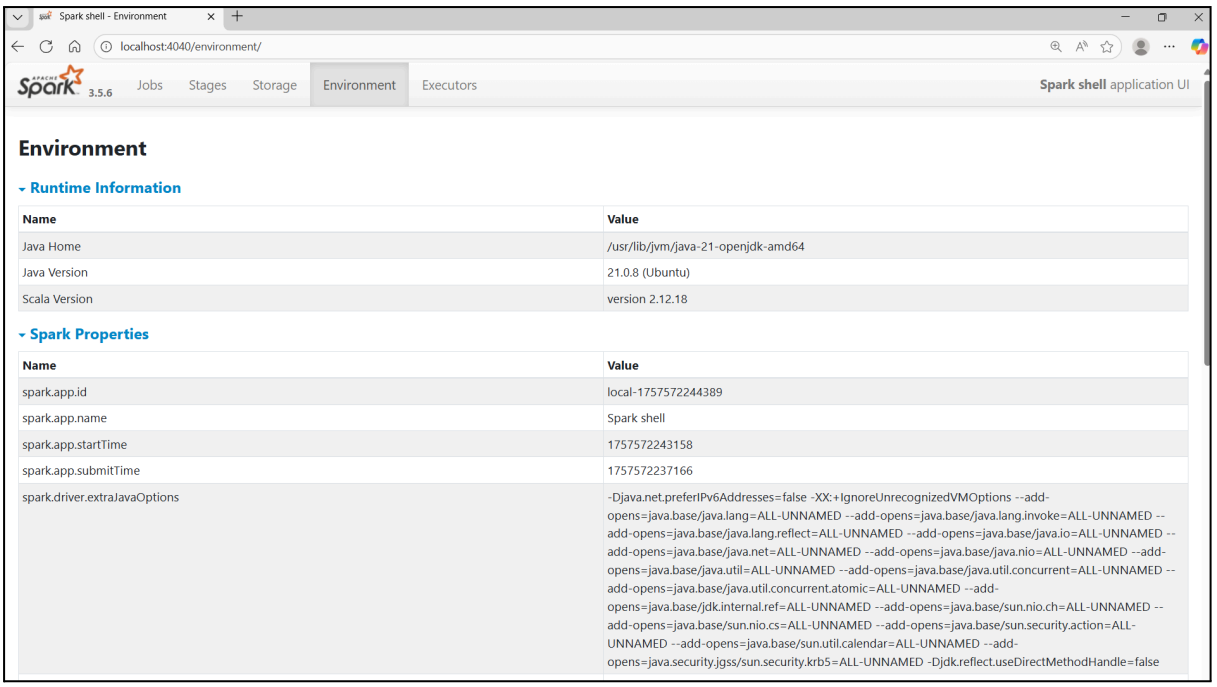
Jobs Tab

This is the Spark Shell Jobs Tab where it shows Spark jobs and their execution stages. It shows all the Spark jobs that have been submitted during the current application run. Each job is broken down into *stages* and *tasks*, and the UI displays their current status (running, succeeded, or failed). This is helpful for troubleshooting performance bottlenecks and ensuring that Spark workloads finish correctly. (Apache Spark, 2025)



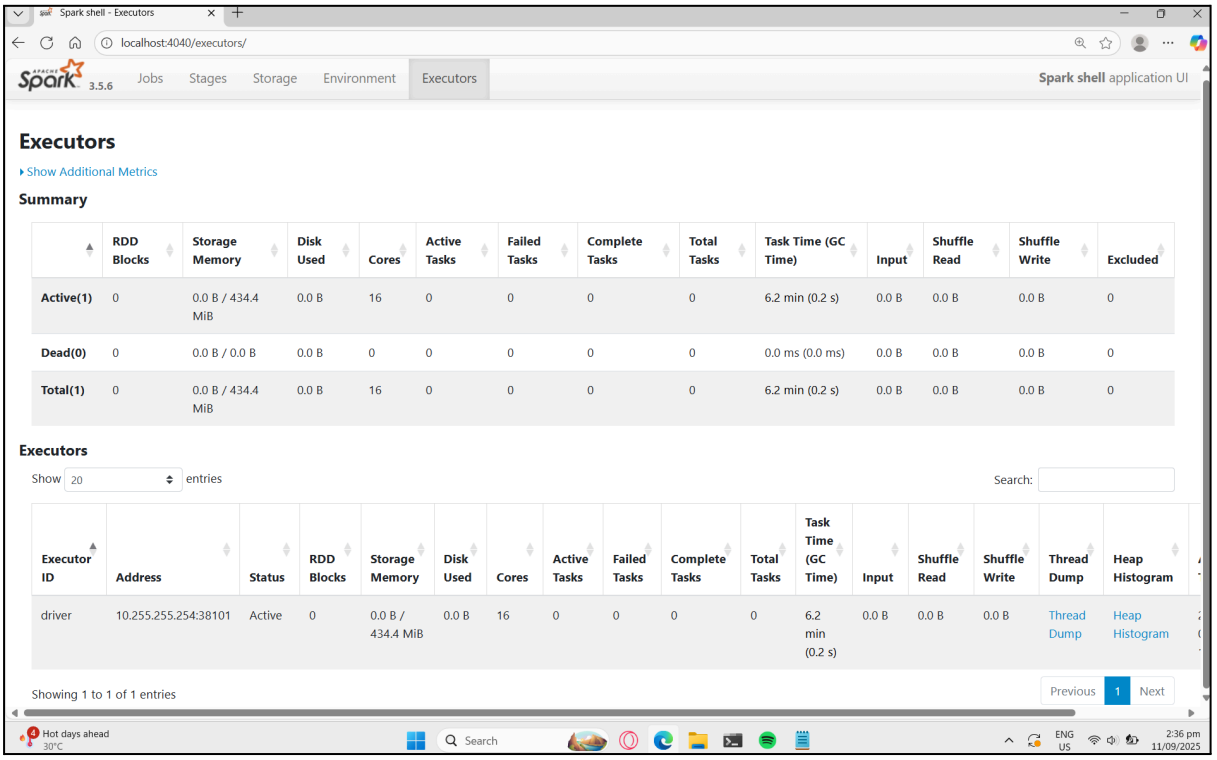
Environment Tab

This is the Spark Shell Environment Tab where it displays runtime configurations. It shows Spark properties, Hadoop properties, system environment variables, and JVM information. Developers often use this tab to confirm configuration settings or detect misconfigurations (Apache Spark, 2025).



Executors Tab

This is the Spark Shell Executors Tab where it lists worker nodes and their allocated resources. It displays metrics such as active tasks, memory usage, and storage consumption. Monitoring this tab helps optimize cluster performance by balancing workload and resource usage (Apache Spark, 2025).



References

Apache Kafka. (2025, September). *Apache Kafka Releases*. Apache Kafka. Retrieved September 11, 2025

Apache Spark. (2025, September). *Jobs tab*. In *Monitoring and Instrumentation*. Apache Spark. Retrieved September 12, 2025

Apache Spark. (2025, September). *Environment tab*. In *Monitoring and Instrumentation*. Apache Spark. Retrieved September 12, 2025

Apache Spark. (2025, September). *Executors tab*. In *Monitoring and Instrumentation.* Apache Spark. Retrieved September 12, 2025

GeekforGeeks. (2024, April 23). *Wget Command in Linux/Unix*. GeeksforGeeks. Retrieved September 11, 2025

GeekforGeeks. (2025, April 11). *How to Compress Files in Linux | Tar Command*. GeeksforGeeks. Retrieved September 11, 2025

GeeksforGeeks. (2025, July 11). *Command in Linux with Examples*. GeeksforGeeks. Retrieved September 11, 2025

GeeksforGeeks. (2025, August 29). *curl Command in Linux with Examples*. GeeksforGeeks. Retrieved September 12, 2025

Gerardi, R. (2022, August 27). *A beginner's guide to tmux*. Red Hat. Retrieved September 11, 2025

Verma, J., Kurup, M., & guide, s. (2025, June 22). *What Is .bashrc in Linux? Configuration Explained*. DigitalOcean. Retrieved September 11, 2025